САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчёт

Лабораторная работа № 6
Настройка системы мониторинга ресурсов

Выполнил:

Ермаков Максим Олегович

Группа K3340

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Настроить в Grafana мониторинг приложения: показать использование CPU, диска, оперативной памяти и сети ВМ, доступность всех микросервисов и журналы работающих контейнеров. Из дополнительных заданий выполнить почтовое оповещение об изменении healthcheck.

Ход работы

1 Выбор сервера и компонентов мониторинга

Для работы я использовал сервер с адресом 31.77.133.132 и Ubuntu 24.04. На нём уже развёрнуто приложение бронирования столиков из ЛР4. Это позволило наблюдать за настоящими микросервисами, PostgreSQL и RabbitMQ.

Приложение находится в /opt/lab4. Мониторинг размещён отдельно в /opt/lab6-monitoring и запускается своим Docker Compose-проектом. Такое расположение позволяет обновлять приложение через существующий GitHub Actions, не перезаписывая настройки мониторинга.

Для числовых показателей используются Node Exporter, Blackbox Exporter и Prometheus. Node Exporter предоставляет показатели Linux, Blackbox выполняет HTTP-проверки, а Prometheus сохраняет результаты и их историю. Grafana запрашивает эти данные и строит панели.

Журналы собираются другой цепочкой: Alloy читает stdout и stderr контейнеров через Docker API, передаёт сообщения в Loki, а Grafana показывает их по запросу.

Повторяющиеся действия подготовки и проверки вынесены во вспомогательные Python-скрипты. После запуска постоянный сбор данных выполняют контейнеры мониторинга

2 Развёртывание и доступ к Grafana

В compose.yml описаны шесть компонентов мониторинга, их сети, тома и лимиты памяти. Версии образов закреплены. Для запуска из каталога мониторинга использовались следующие команды:

```
cd /opt/lab6-monitoring
python3 scripts/prepare.py --alert-email \
  maksimermakov1337@gmail.com
docker compose config -q
docker compose pull
docker compose up -d
docker compose ps
```

prepare.py создал случайный пароль Grafana, закрытые файлы секретов и .env. Команда config -q проверяет Compose без печати подставленных значений, pull скачивает готовые образы, up -d запускает контейнеры в фоне, ps показывает их состояние.

Пароли находятся только в серверной папке secrets и передаются Grafana как файлы Docker secrets. Рабочий .env и secrets исключены из Git. Опубликованные порты Grafana, Prometheus и Loki привязаны к 127.0.0.1, поэтому для доступа я использую SSH-туннель:

```
ssh -i ~/.ssh/it_garage -o IdentitiesOnly=yes \
  -o ExitOnForwardFailure=yes -N \
  -L 127.0.0.1:13000:127.0.0.1:3000 root@31.77.133.132
```

Туннель пересылает порт 13000 на MacBook к порту 3000 сервера. Дашборд открывается по адресу http://localhost:13000/d/lab6-it-garage. Для входа используется admin и случайный пароль. После закрытия туннеля просмотр недоступен, но сервер продолжает собирать данные и отправлять письма.

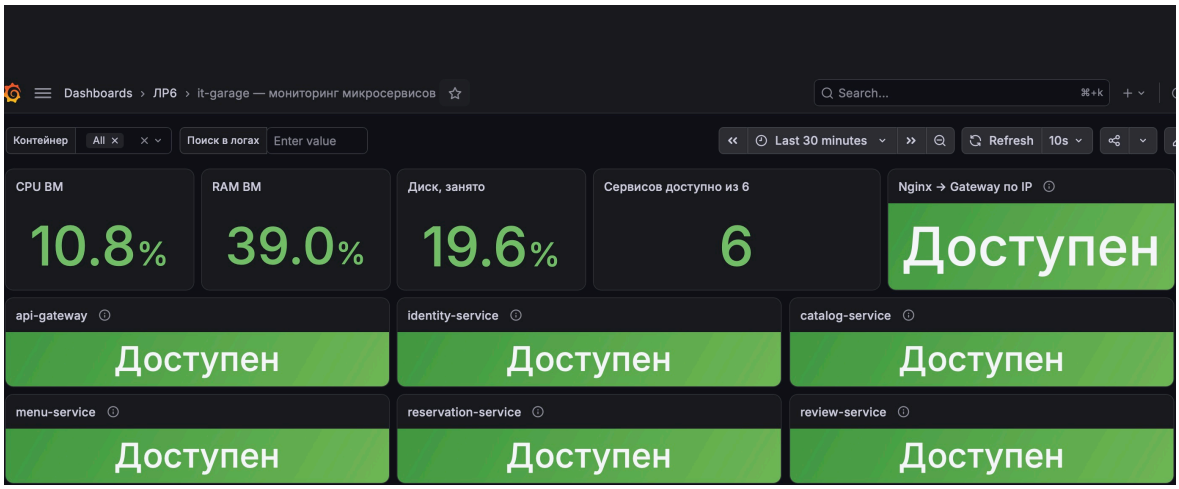


Рисунок 1. Готовый дашборд и шесть доступных микросервисов

3 Мониторинг процессора и оперативной памяти

Node Exporter настроен на ресурсы самой ВМ: ему доступны пространства сети и процессов хоста, а корневая файловая система подключена для чтения. Проверка показала 2 vCPU и около 3,8 ГиБ RAM. Prometheus получает показатели каждые 15 секунд.

Загрузка CPU рассчитывается по изменению счётчика времени простоя. Функция rate определяет изменение за секунду, затем доля простоя вычитается из единицы и переводится в проценты. График позволяет увидеть нагрузку за выбранный период.

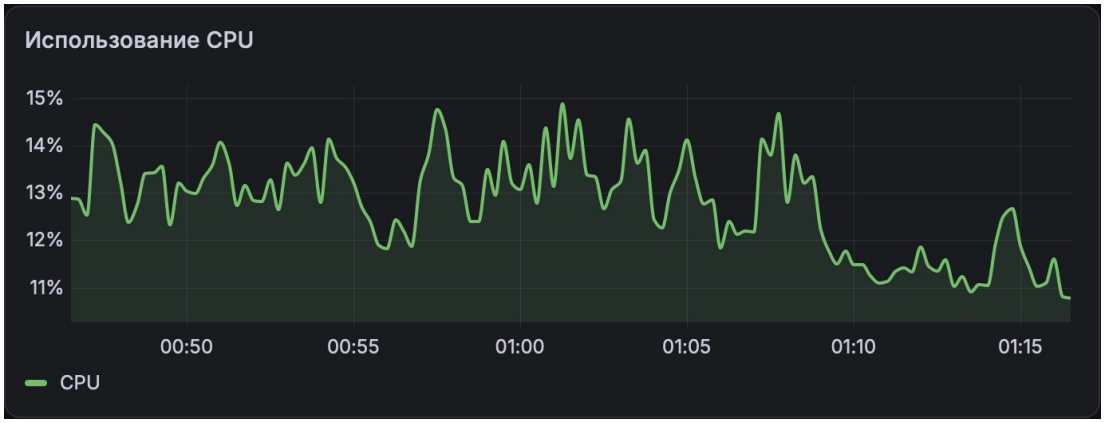


Рисунок 2. История использования процессора ВМ

Для памяти используются MemTotal и MemAvailable. Вторая метрика учитывает память, которую Linux может предоставить приложениям, в том числе после освобождения части кеша. На графике показаны используемый и доступный объёмы.

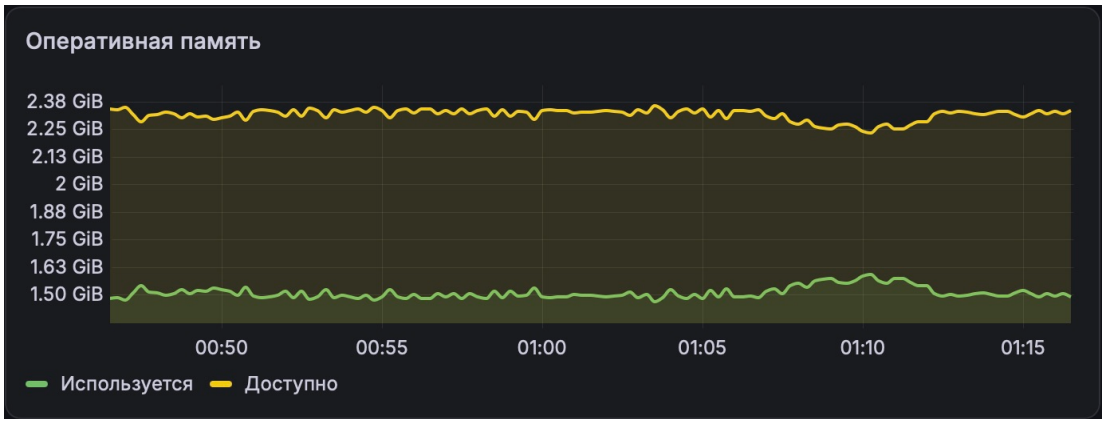


Рисунок 3. Используемая и доступная оперативная память

4 Мониторинг диска и сети

Для корневой файловой системы отображаются размер и доступное место. На графике зелёным показано занятое или недоступное обычному пользователю пространство, жёлтым - доступное. Отдельная панель дашборда показывает скорость чтения и записи в байтах в секунду.

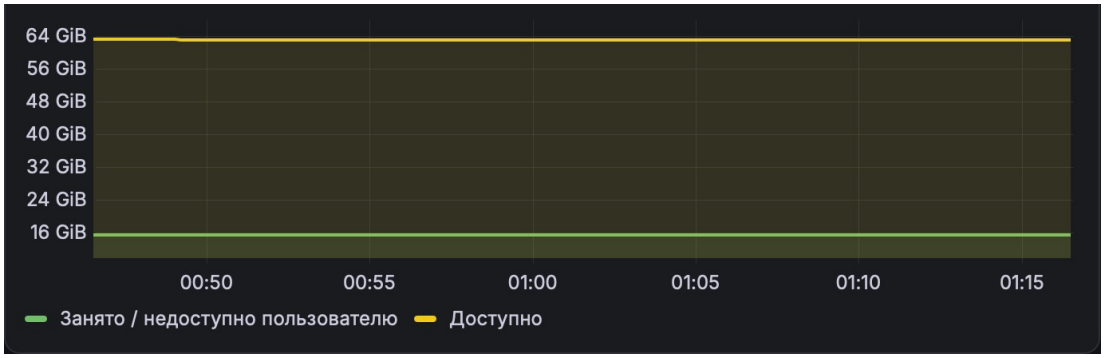


Рисунок 4. Занятое и доступное место в корневой файловой системе

Сетевой график строится по счётчикам принятых и отправленных байтов с помощью rate. Поэтому он показывает скорость обмена, а не суммарный трафик с момента запуска. Из запроса исключены loopback и внутренние интерфейсы Docker, чтобы не учитывать один обмен несколько раз.

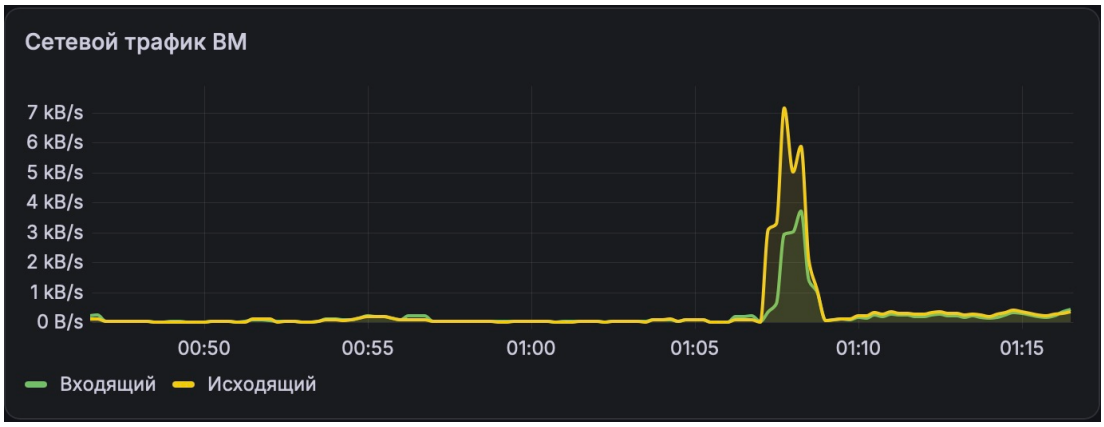


Рисунок 5. Входящий и исходящий трафик сетевого интерфейса ВМ

В Prometheus история ограничена сроком 7 дней и объёмом 2 ГБ. При достижении ограничений старые блоки удаляются. Grafana обновляет экран каждые 10 секунд; это отдельный интервал, который не меняет частоту сбора метрик.

5 Проверка доступности микросервисов

В prometheus/prometheus.yml перечислены адреса /health Gateway, identity-service, catalog-service, menu-service, reservation-service и review-service. Blackbox подключён к внутренней сети lab4_default и обращается к сервисам по Docker-именам. Открывать их порты в Интернет не требуется.

В blackbox/blackbox.yml заданы GET, тайм-аут 5 секунд и два условия успеха: HTTP 200 и наличие status: ok в теле ответа. Результат сохраняется в метрике probe_success: 1 означает успех, 0 - отказ. Для счётчика доступных сервисов используется запрос:

```
sum(probe_success{job="microservices_health"})
```

Дополнительно проверяется http://31.77.133.132/health через Nginx. Этот запрос выполняется с той же ВМ. Наш /health подтверждает доступность HTTP-процесса, но не выполняет полную проверку БД, RabbitMQ или бронирования. Поэтому зелёные индикаторы имеют именно такой смысл.

6 Сбор и просмотр журналов контейнеров

В alloy/config.alloy настроены обнаружение контейнеров, чтение сообщений и отправка в Loki. Метки container, service и project позволяют выбрать нужный источник. Loki хранит журналы в постоянном томе; срок хранения задан как 72 часа, удаление старых записей выполняется асинхронно.

В Grafana я выбрал контейнер menu-service и период за последние сутки. В журнале видны настоящие сообщения запуска, в том числе после проверки восстановления. Строки передаются из stdout и stderr; если сервис молчит, новых сообщений в выбранном периоде может не быть.

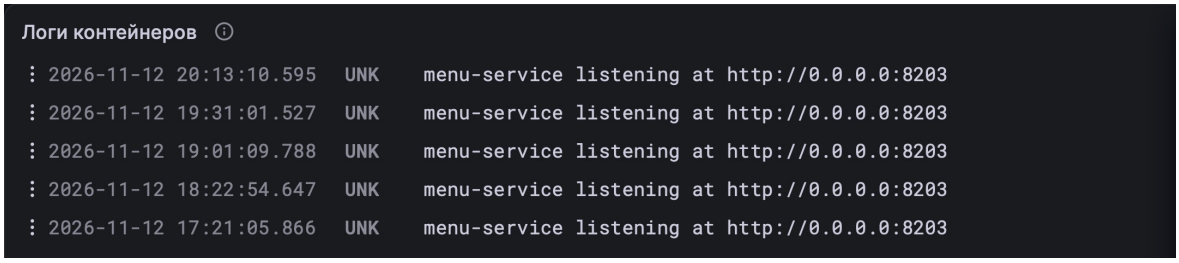


Рисунок 6. Сообщения запуска menu-service в панели журналов

При первоначальной проверке были получены сообщения всех 14 контейнеров: восьми контейнеров приложения и шести компонентов

мониторинга. Сбор распространяется также на PostgreSQL и RabbitMQ. Содержимое сообщений и исходные отметки времени сохраняются.

7 Настройка оповещения и почтовой отправки

Источники данных, дашборд, правило и почтовый контакт загружаются Grafana из папки provisioning. Это позволяет воспроизвести настройку из файлов. Скрипт build-dashboard.py формирует JSON панелей и правила, но не участвует в постоянном сборе данных.

Правило «Микросервис недоступен» получает probe_success всех шести сервисов, берёт последнее значение и проверяет условие меньше 1. Оно вычисляется каждые 20 секунд. Параметр for: 60s требует устойчивого отказа в течение минуты и защищает от уведомлений о кратком сбое.

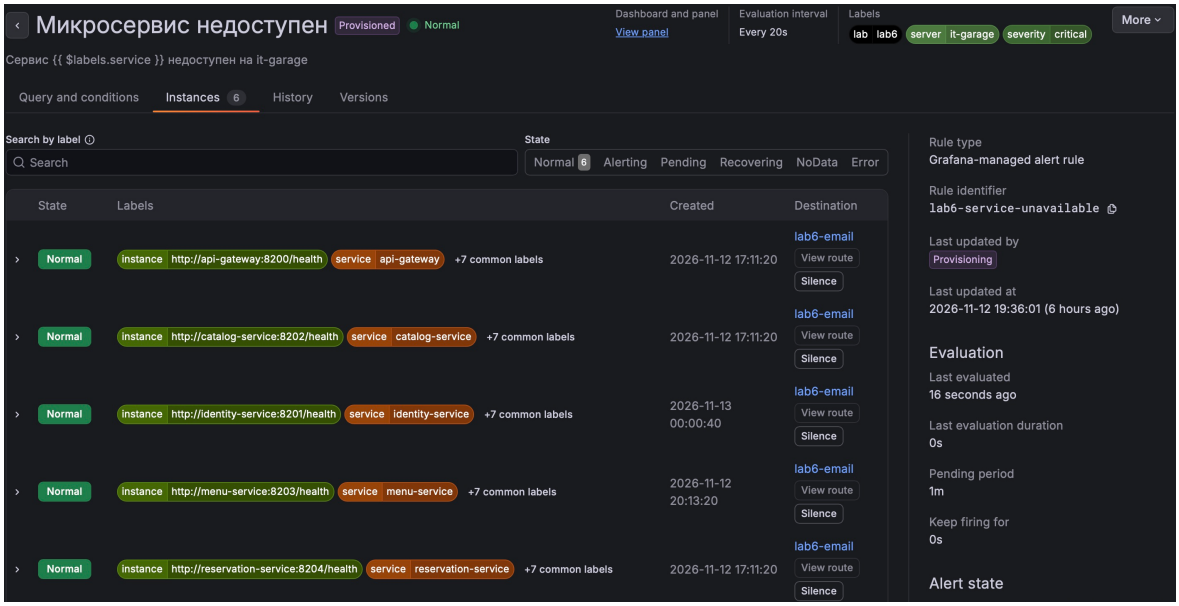


Рисунок 7. Правило Grafana с интервалом 20 секунд и ожиданием 1 минуты

Для отправки выбран Gmail SMTP smtp.gmail.com:587 с обязательным STARTTLS и проверкой сертификата. Пароль приложения Google я ввёл скрыто через configure-gmail.py. Скрипт проверил вход и сохранил пароль в закрытом серверном файле. Для применения параметров Grafana была пересоздана:

```
docker compose up -d --force-recreate grafana
```

Контакт lab6-email отправляет письма на maksimermakov1337@gmail.com. В notifications.yml разрешены сообщения об отказе и восстановлении. Начальное ожидание отправки составляет 10 секунд, повторное напоминание о продолжающейся проблеме - 4 часа.

8 Проверка отказа и восстановления

Для проверки использован настоящий отказ menu-service 12 ноября 2026 года. Проверочный сценарий по SSH остановил сервис, дождался состояния Alerting и затем запустил его. Запуск находился в блоке finally, чтобы выполнить восстановление даже при ошибке проверки. Использовались команды Docker Compose:

```
docker compose --project-directory /opt/lab4 stop menu-service
docker compose --project-directory /opt/lab4 start menu-service
```

Остановка завершилась в 18:59:22 по Москве. Очередной healthcheck дал probe_success=0, индикатор menu-service стал красным, а число доступных сервисов уменьшилось до 5. Правило перешло в Pending, затем в 19:00:43 было зафиксировано Alerting.

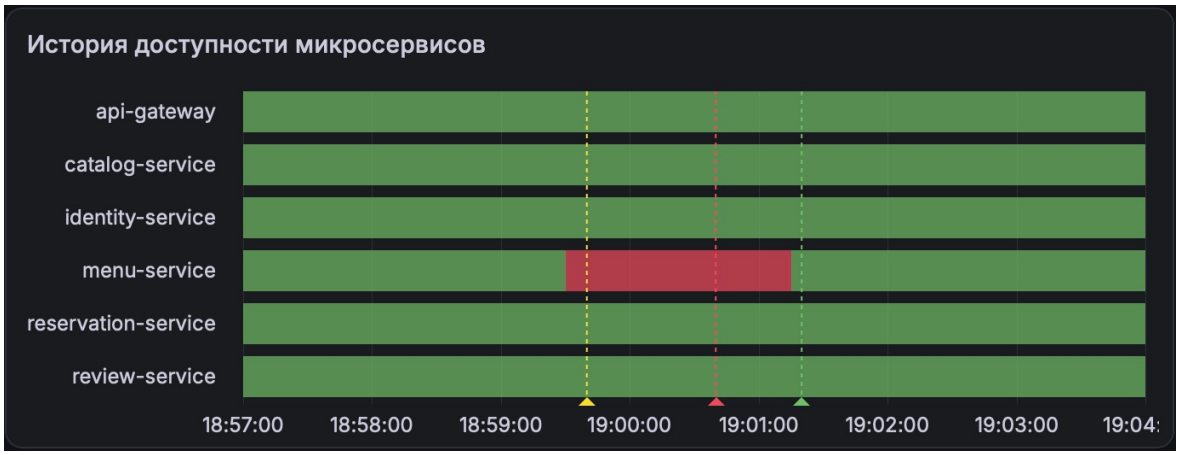


Рисунок 8. Сохранённая история отказа menu-service 12 ноября

Сервис был запущен в 19:01:09. После успешного /health правило вернулось в Normal в 19:01:24. На графике красный участок сменяется зелёным, а остальные сервисы продолжают проходить проверки. Для иллюстрации выбран период с 18:57 до 19:04 по Москве.

Письмо появляется позже красного индикатора: сначала Prometheus собирает очередные данные, затем правило должно подтвердить минуту отказа, после чего требуется время на отправку. Если сервис восстановится до завершения Pending, такое краткое событие не приведёт к обычному письму firing.

Мониторинг обнаруживает отказ и сообщает о нём, но сам не запускает микросервис. В этой проверке восстановление выполнил проверочный

сценарий. Для ручной демонстрации можно использовать те же команды stop и start.

9 Получение почтовых уведомлений

На мою почту пришло уведомление firing в 19:00 и resolved в 19:01. В теме и тексте указаны сервис и его состояние.

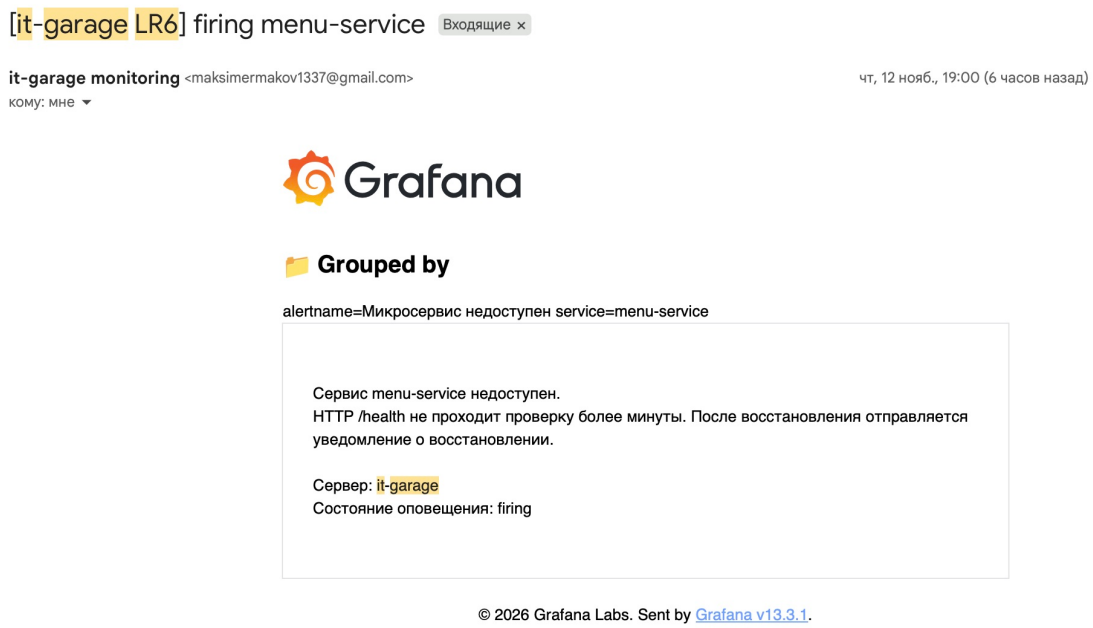


Рисунок 9. Письмо о недоступности menu-service в 19 часов

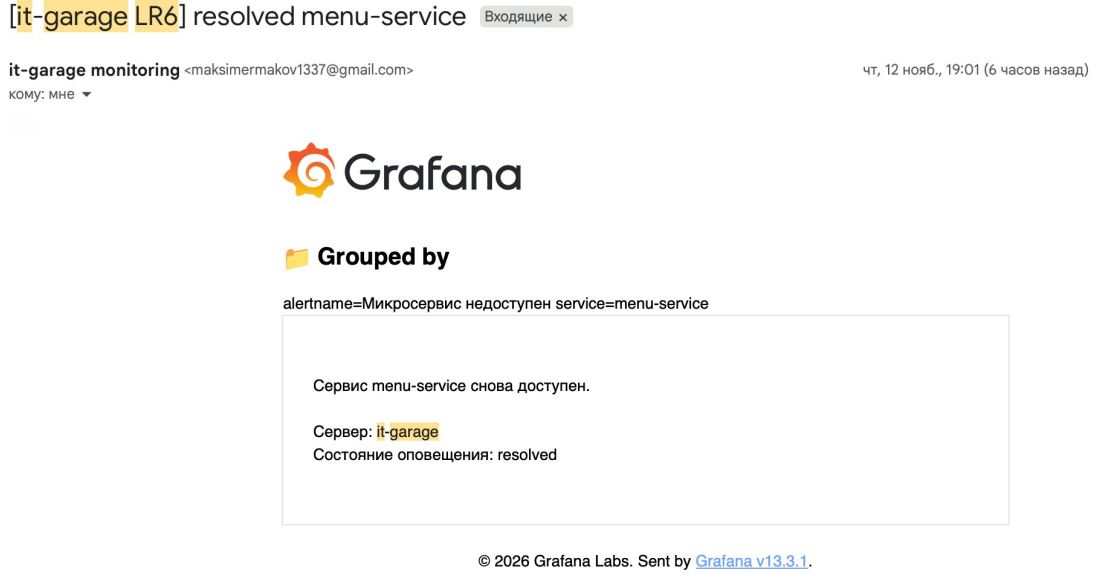


Рисунок 10. Письмо о восстановлении menu-service в 19 часов 1 минуту

Получение обоих автоматических писем подтвердило полный путь от изменения healthcheck до доставки уведомления. Скриншоты показывают уже полученные сообщения в Gmail.

Вывод

В работе я использовал Grafana для наблюдения за ресурсами ВМ, доступностью микросервисов и журналами контейнеров. Prometheus сохраняет числовые показатели, а Loki - сообщения. Реальная остановка menu-service отразилась в истории и вызвала письмо firing; после запуска пришло resolved. Основное задание и дополнительное задание с почтовыми уведомлениями выполнены.